



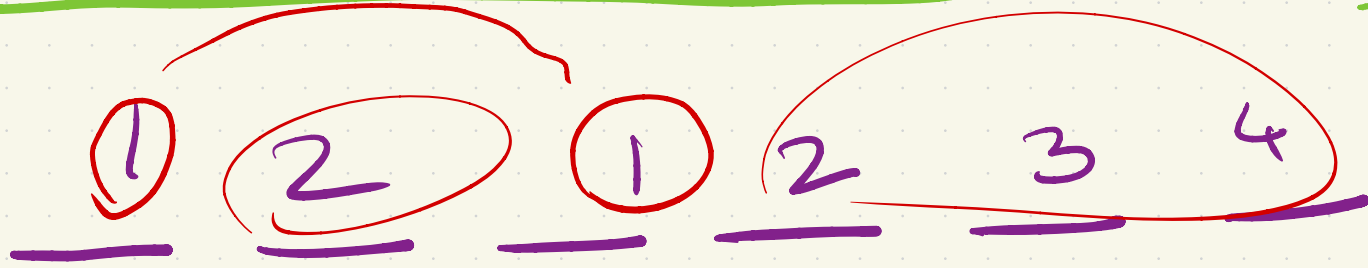
* 3 DP Problems

Dynamic Programming Problem Solving 3

- Gaurish Baliga

Additional Task

GOOD
Try!!



1 move 2

Remove any subarray that is
a palindrome

min moves $\rightarrow$ to make array empty

Problem 1

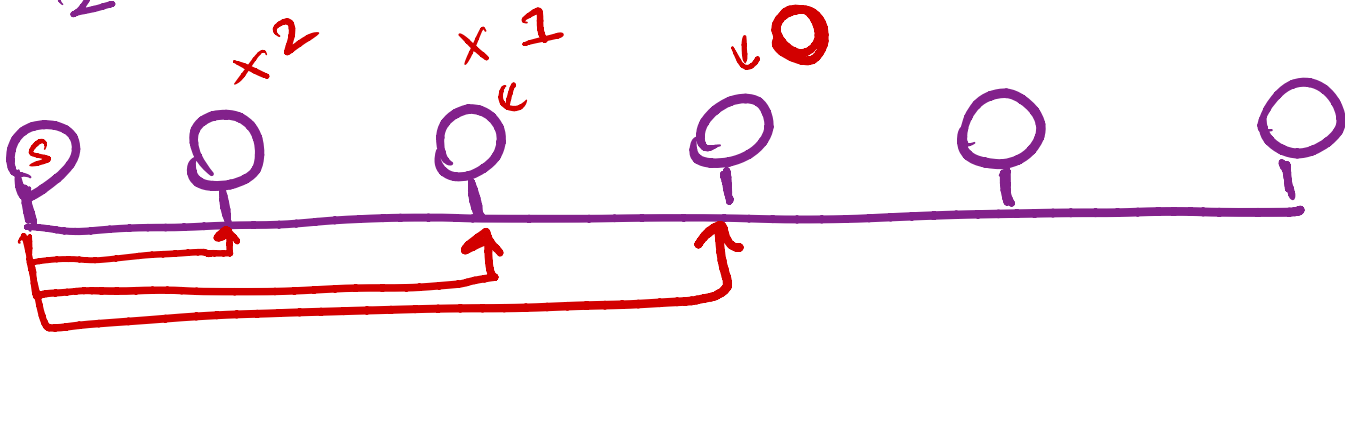


Road Optimization ↙

<https://codeforces.com/problemset/problem/1625/C>

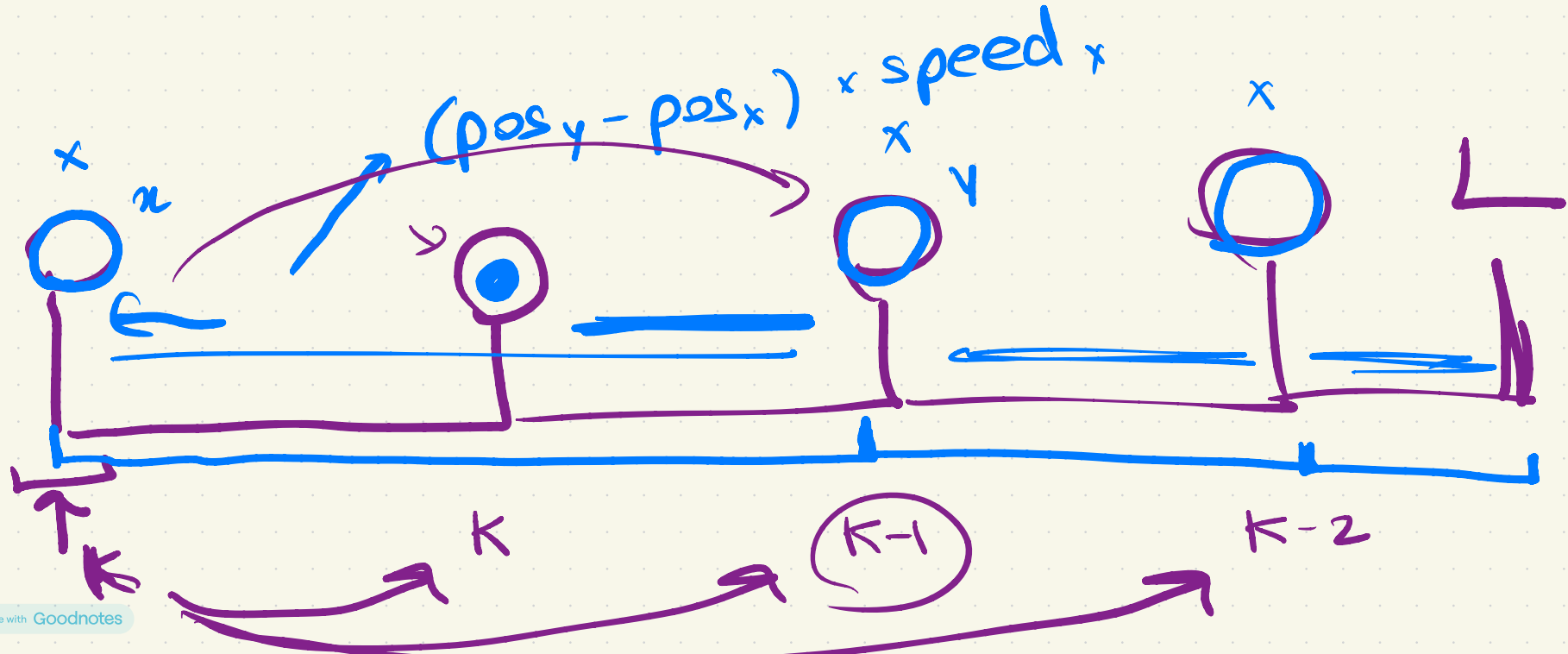
Biggest hint

2



n^3 space
X MLE

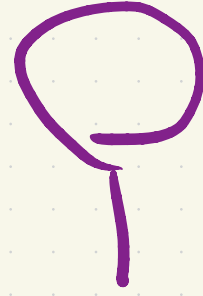
$k \rightarrow$ skip atmost k signs



$j \in [i+1, n-1]$



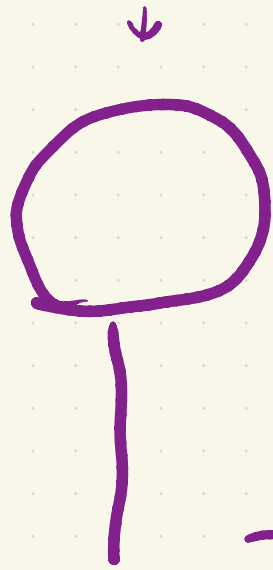
next flagpost



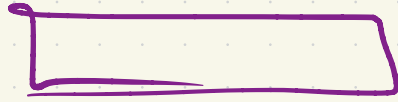
flagpost i

flagpost j



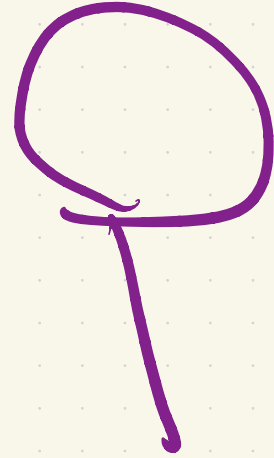


$$i < j$$



$$j < n$$

$$(pos_j - pos_i) \times speed_i$$



flagpost

i

flagpost

j

k

$\leftarrow$

$k - (j - i - 1)$

skipped

$$\underline{j} < n \quad \&\& \quad \underline{k - (j - i - 1)} \geq 0$$

↳ choices for the next speed post

$$\underline{j = i + 1 \quad ; \quad j < n \quad \& \quad k - (j - i - 1) \geq 0}^{jth}$$

$dp_{i \rightarrow k}^{n \rightarrow n}$ $\rightarrow$ min cost required
to get to L from

$O(n^2)$

pos: such that
we are fixing
flagpost i &

we can skip
flagpost

$$\underline{dp_{i,k}} = \min ((pos_j - pos_i) \times speed;$$

state?

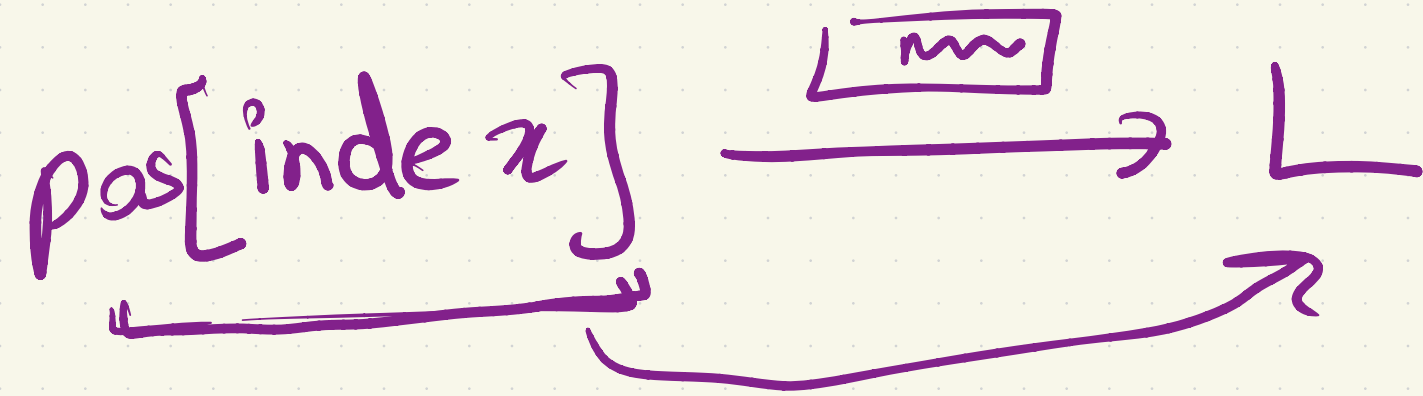
transitions?

$$+ dp[j][k - (j - i - 1)]$$

over all possible j

$$\underline{dp_{i,j,k}} \quad \begin{matrix} \swarrow i \\ \nwarrow prev \\ \searrow k \in \text{left} \end{matrix}$$

$$\underline{O(n^3) \text{ states}} \quad \text{10}^8 \text{ space}$$



adding a
↓ virtual
2 element

```
int n, l, k; cin >> n >> l >> k;

vector<int>pos(n + 1), speed(n + 1); pos[n] = l;

for(int i = 0; i < n; i++) cin >> pos[i];
for(int i = 0; i < n; i++) cin >> speed[i];

vector<vector<int>>dp(n, vector<int>(k + 1, -1));
cout << f(0, k, pos, speed, dp) << "\n";
```

0, *

TC $\rightarrow O(n^3)$
SC $\rightarrow O(n^2)$

```
const int INF = 1e9 + 1;
```

```
int f(int index, int leftToSkip, vector<int>&pos, vector<int>&speed, vector<vector<int>>&dp) {
```

```
    if(index == speed.size() - 1) return 0; pos = L
```

```
    if(dp[index][leftToSkip] != -1) return dp[index][leftToSkip];
```

```
    int answer = INT_MAX;
```

```
    for(int i = index + 1; i < speed.size() && (i - (index + 1)) <= leftToSkip; i++) {
```

```
        answer = min(answer, f(i, leftToSkip - (i - (index + 1)), pos, speed, dp) + (pos[i] - pos[index]) * speed[index]);
```

```
    }
```

```
    return dp[index][leftToSkip] = answer;
```

```
}
```

cost for travel

$(j - i - 1) \rightarrow (j - (i + 1))$

$dp[i][j][k]$ space optimised.

take post

not take

$+ (pos_{i+1} - pos_i) \times$
speed_i

$dp[i+1][j][k]$

$dp[i+1][j][k-1] + (pos_{i+1} - pos_i) \times$
speed_j

iterative

$i \rightarrow i+1$

$i, j \rightarrow j, k \geq j$

$k \rightarrow k-1$

$dp[j][k]$
 $\rightarrow ndp[j][k]$

$i = n-1; i \geq 0; i--$
decreasing

decreasing

increasing



space opt

$dp[n][k] \rightarrow$ calculated

$ndp[n][k]$ $\rightarrow$ calculating

Problem 2



Clear the String

<https://codeforces.com/contest/1132/problem/F>

a b a c a → a a a → 1 move

3 moves

→ empty → 3

↓ ↓ ↓
a b a c a
→

new technique

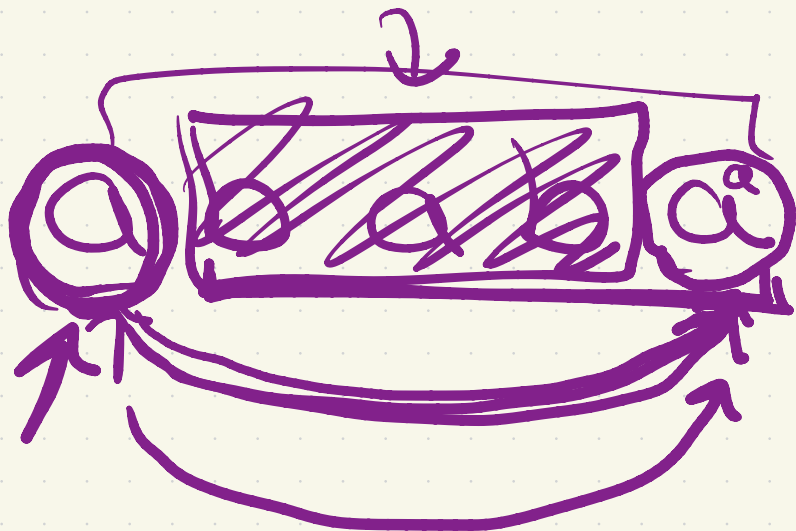
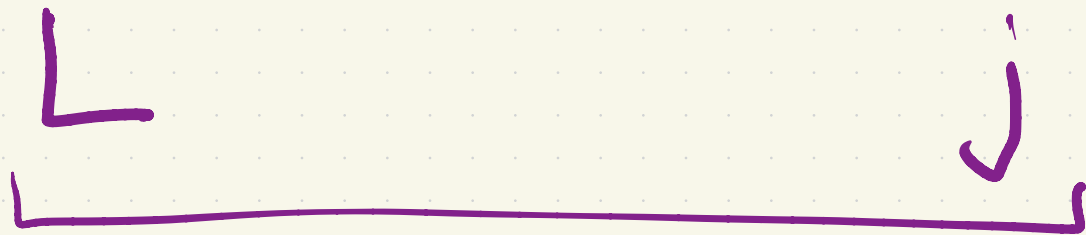
remove entire string
pieces of the string

Base case → 1 character → 1 move

↓
→ (a) b a
L j

←
c c a
R

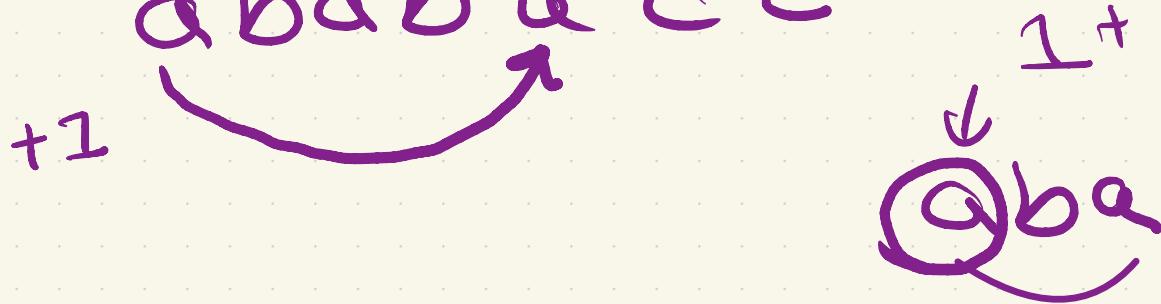
3



ababab|ce



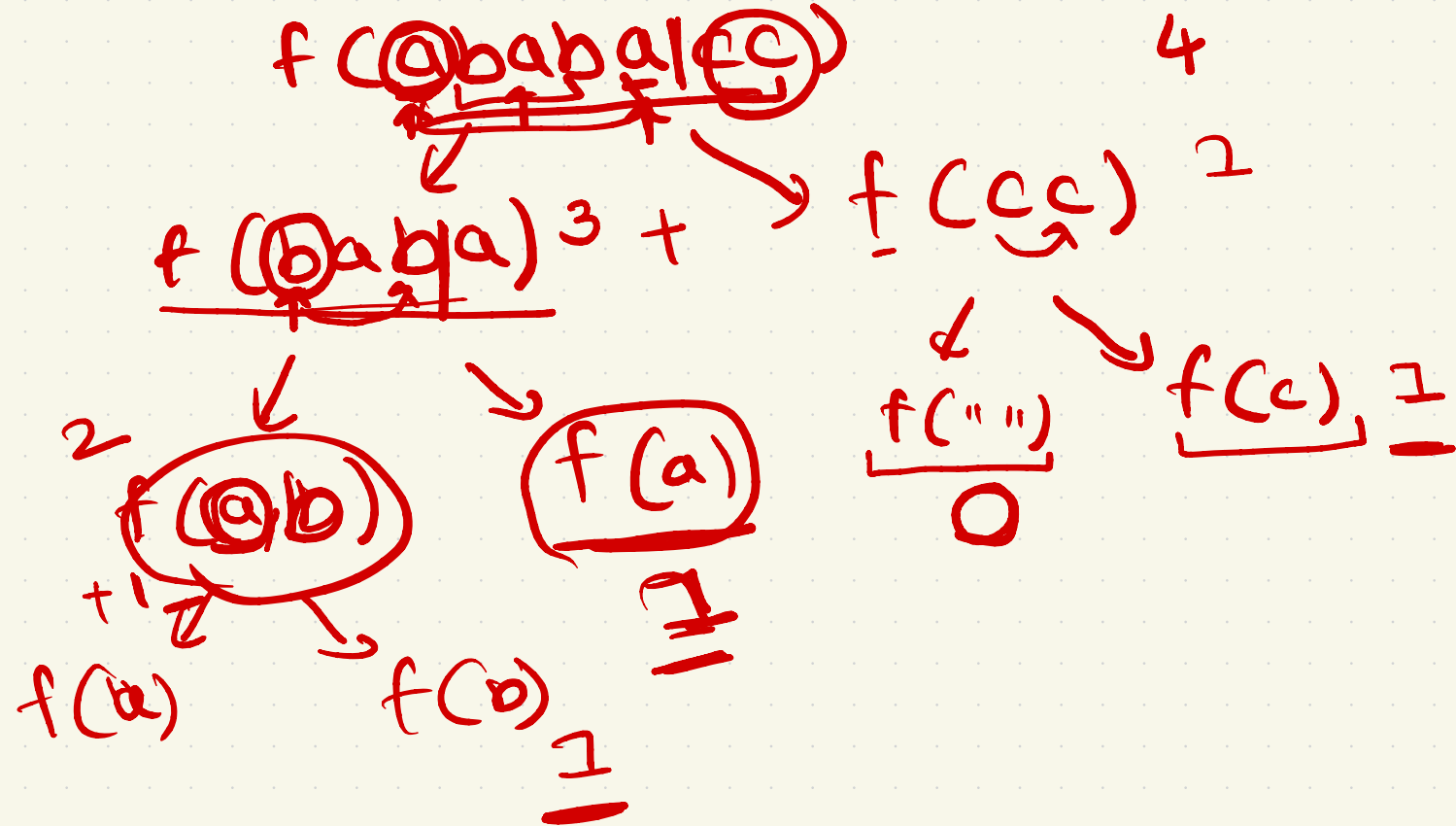
abababcc



$a b \overset{+1}{\circ} a \overset{+1}{\circ} b \overset{+1}{\circ} a \overset{+1}{\circ} c c = 4$

$f(abab|cc)$

$\searrow f(cc)$



ababab

abab

L i-1

L+1 i

$$f(" ") = 0$$

$$f(L, R) = L = R = 1$$

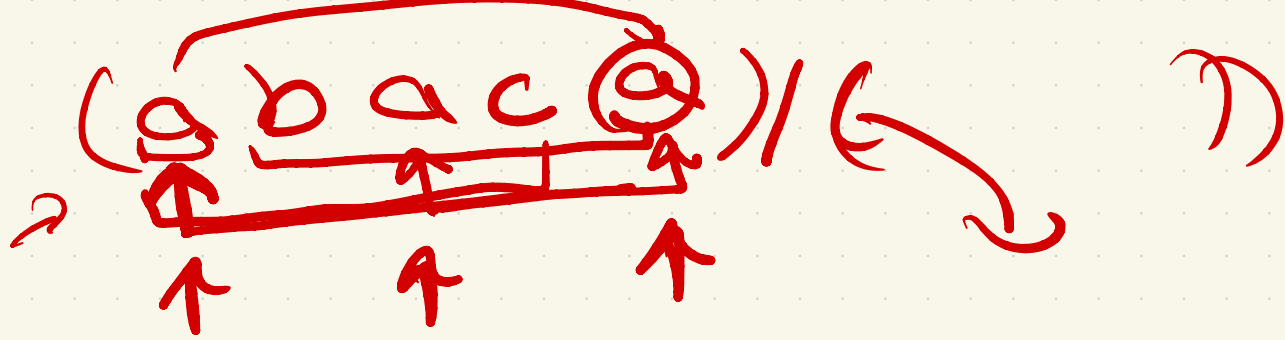
$$f(L, R) =$$

$$\min \left(f(L+1, i) + f(i+1, R) \right)$$

$$S_L = S_i \quad i > L$$

PL

$$[1 + f(L+1, R)]$$



1. associate first char to
last same

2. associate last char to
first same

3. associat first char to
next same character

$f(\underline{a} \text{ } \textcircled{b} \text{ } \textcircled{c} \text{ } \textcircled{d})$

→ fix the next character in the chain

$f(L, R) \quad i > L$

$f(L+1, i-1) + f(i, R)$

aba ca

(151)
b + aca

new

$$\underline{f(L, r)} = \begin{cases} \underline{1 + f(L+1, r)} \end{cases}$$

min

$$\begin{aligned} \underline{L=r} &= \underline{1} \\ \underline{L > \emptyset} &= \underline{0} \end{aligned}$$

$$\begin{aligned} &\min (f(L+1, i-1) \\ &\quad + f(i, R)) \end{aligned}$$

$$\begin{aligned} \underline{S_L} &= \underline{S_r} \\ \underline{i > L} \end{aligned}$$

$f(L, r) \rightarrow$ chose an index between
 L to r . (j).

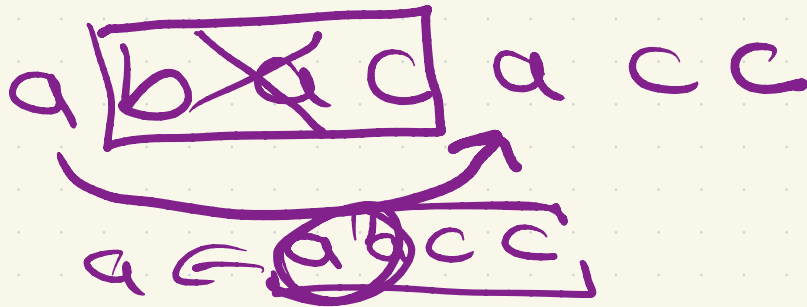
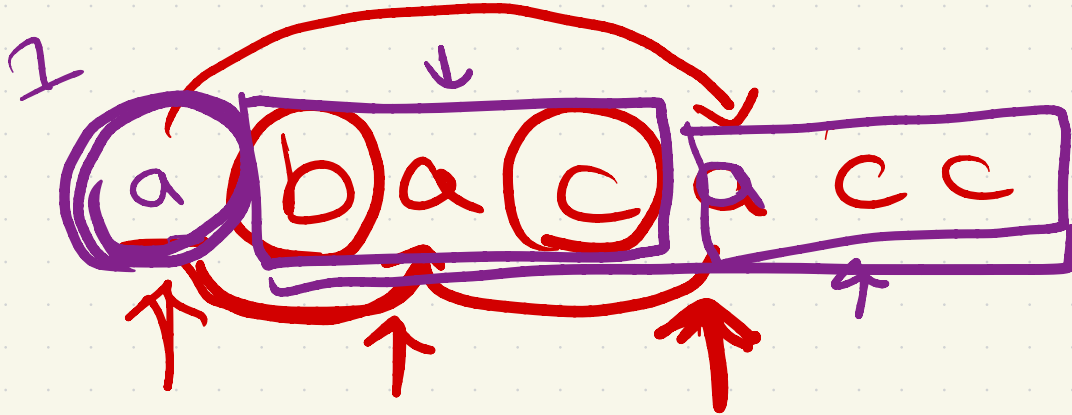
a b a b a
L j

j+1 r
disjoint

$$\underline{f(L, r)} = f(j+1, r) + \underline{\text{something}}$$

min cost to remove from L to j

$[0, n-1]$



idea



2000

$S_L \rightarrow$ next character in chain

```
int f(int left, int right)
{
    if(left > right) return 0;
    if(dp[left][right] != -1) return dp[left][right];
    int answer = 1 + f(left + 1, right);
    for(int i = left + 1; i <= right; i++)
    {
        if(s[left] == s[i]) answer = min(answer, f(left + 1, i - 1) + f(i, right));
    }
    return dp[left][right] = answer;
}
```

in between

next chain

fix next character

Problem 3

Cunning Gena

<https://codeforces.com/problemset/problem/417/D>

5 more minutes

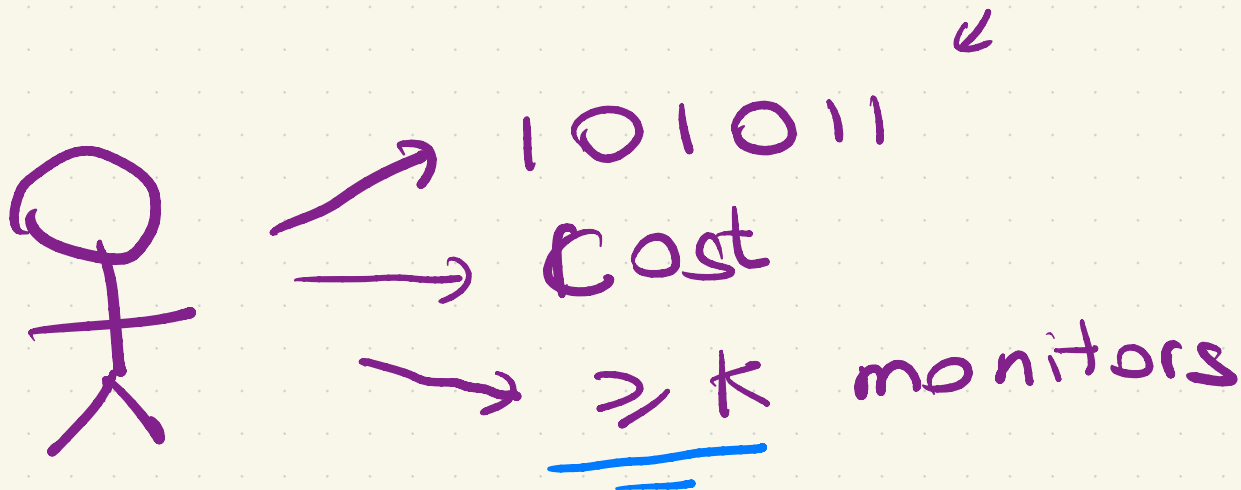


3 ideas → DP / greedy bitmasking
→ space opt

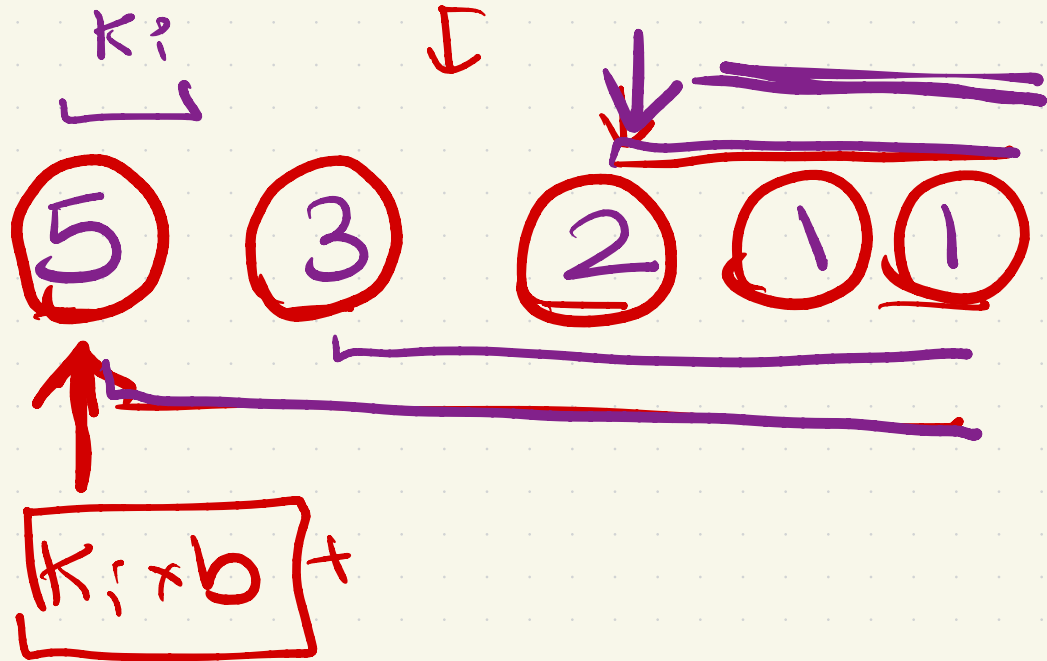
[→ $m \leq 20$ → bitmask - 111111
→ solve for each k
 $\underbrace{k * b}_{\text{monitors}}$

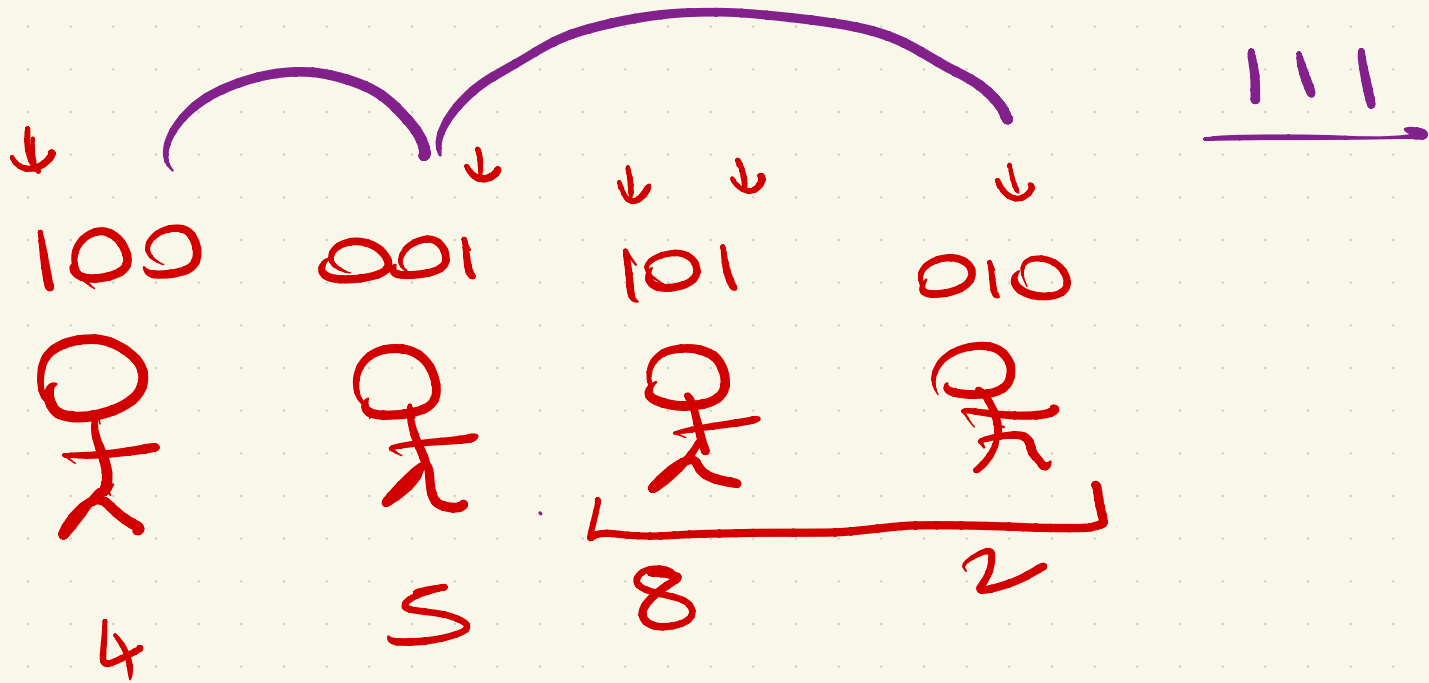


$$n \leq 100$$



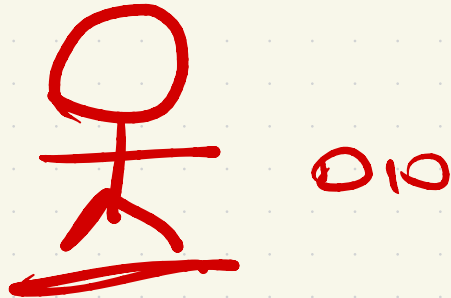
→ We can sort the people
on descending order of k





Bitwise OR = 1111...

└ 101 = mask



take

note take

$f(i+1, \text{mask})$

C_i

$+ f(i+1, \text{mask} | \text{prob}_i)$

~~inf~~ = n



mask = 111...

↳ 0

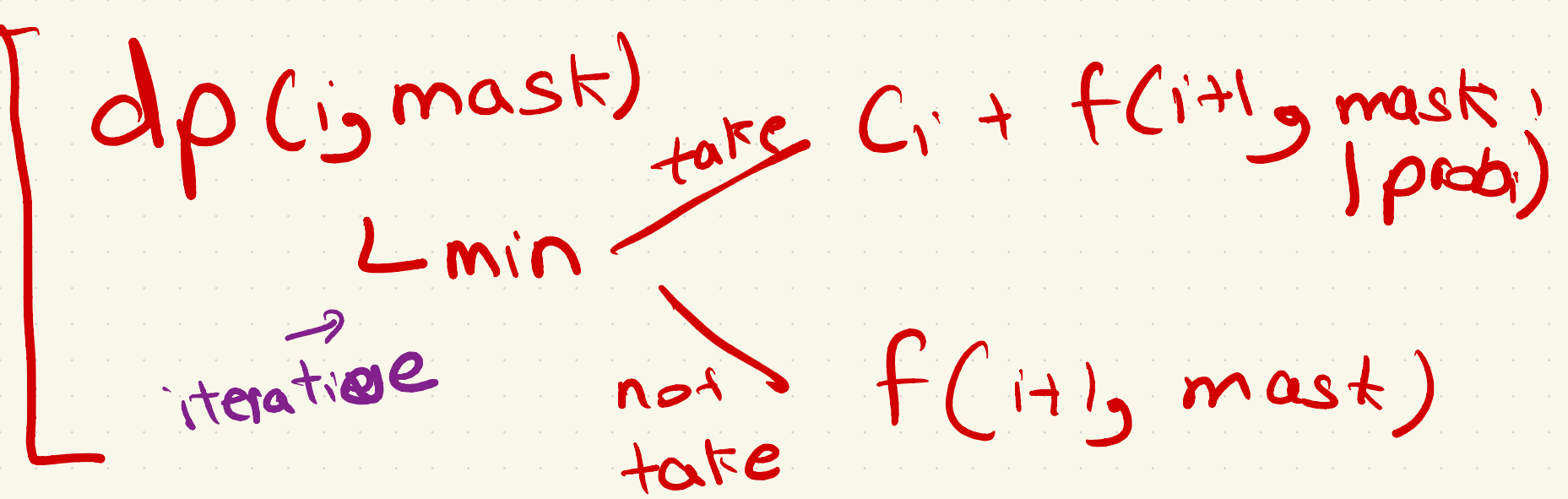
mask $\neq$ 111

↳ INF ∞

$dp_{i, \text{mask}}$

$\rightarrow$ min cost to solve problems such that we are solving

$[i \dots n-1]$ persons and the problems we can already solve = mask



sorted people based
on k $O(n \times 2^n)$

→ for every k $O(k \times n \times 2^m)$

~~X~~ separate dp table??

$O(n \times 2^m)$

$dp(1,0) + cost$



4

$dp(3,0)$



1

3

5



$dp(0,0) + cost$



$dp(2,0) + cost$

$$n \times 2^m \rightarrow 100 \times 2^{20} = 1e8$$

~~X~~ array of size 1e8



```
int n, m, b; cin >> n >> m >> b;

vector<person> arr(n);
for (auto &p : arr) {
    cin >> p.cost >> p.monitor;
    int sz; cin >> sz;
    int tmpMask = 0;
    while (sz--) {
        int x; cin >> x; x--;
        tmpMask |= (1LL << x);
    }
    p.mask = tmpMask;
}

sort(arr.begin(), arr.end(), [&](const person &l, const person &r){
    return l.monitor > r.monitor;
});
```

Recursive Code (MLE)

$n \times 2^m$ space

```
int f(int index, int mask, int m, vector<person>&arr) {  
    if(index == arr.size()) {  
        if(mask == (1 << m) - 1) return 0;  
        else return INF;  
    }  
  
    if(dp[index][mask] != -1) return dp[index][mask];  
  
    return dp[index][mask] = min(f(index + 1, mask, m, arr),  
                                arr[index].cost + f(index + 1, mask | (arr[index].mask), m, arr));  
}
```



```

int fullMask = (1LL << m) - 1;
int dpCur[1 << 20], dpNxt[1 << 20];
for(int mask = 0; mask < (1 << m); mask++){
    dpNxt[mask] = (mask == fullMask ? 0 : INF);
}

vector<int> best(n, INF);

for(int i = n - 1; i >= 0; i--){
    for(int mask = 0; mask < (1 << m); mask++){
        int costSkip = dpNxt[mask];
        int costTake = (arr[i].cost >= INF ? INF : arr[i].cost) + dpNxt[ mask | arr[i].mask ];
        dpCur[mask] = min(costSkip, costTake);
    }
    best[i] = dpCur[0];
    for(int mask = 0; mask < (1 << m); mask++){
        dpNxt[mask] = dpCur[mask];
    }
}

int answer = INF;
for(int i = 0; i < n; i++){
    if (best[i] < INF) {
        answer = min(answer, best[i] + arr[i].monitor * b);
    }
}

if(answer >= INF) cout << -1 << "\n";
else cout << answer << "\n";

```

Handwritten annotations:

- index n (pointing to `dpNxt` array)
- decr (pointing to `i--`)
- increasing (pointing to `mask++`)
- dp[i, 0] (pointing to `dpCur[0]`)
- answer (pointing to `answer` variable)